

2017 年面向对象程序设计期末考试参考答案

一、选择题

1.

答案：B

vector 当然支持在中间插入元素，只是这种插入通常代价较高，因为后面的元素可能需要整体后移。list 不支持下标随机访问，而 vector 支持，这也是二者的重要区别。

2.

答案：D

一个类并不一定只有一个构造函数。构造函数可以重载，例如默认构造、带参构造、复制构造都可以同时存在。

3.

答案：D

派生类可以访问基类中的 public 成员和 protected 成员，但不能直接访问基类的 private 成员。这一题主要考察继承中的访问控制。

4.

答案：B

运算符重载不能改变操作数个数，不能改变优先级，也不能改变结合性，但可以为自定义类型赋予合适的运算含义。例如 `Str + Str`、`Complex + Complex` 都是典型例子。因此说“不能改变运算符的功能”并不准确。

5.

答案：B/D

这一题是老题里比较容易出现歧义的一题，B 和 D 的说法都不完全准确。

1. 自定义了某些构造函数后，默认构造函数不一定还会自动生成。
2. 复制构造、赋值运算符、析构函数在很多情况下会被编译器合成，但资源管理类往往不能只依赖默认合成版本。

6.

答案：A

析构函数没有参数，也不能重载。它没有返回值类型，连 void 都不能写；函数名也不是与类名完全相同，而是要写成 ~ 类名。函数体中也不一定必须出现 delete，只有类真正管理动态资源时才需要手动释放。

7.

答案: D

派生类当然可以定义新的虚函数。动态绑定、多态都建立在虚函数基础上,这一题考察的是继承体系中虚函数的基本性质。

8.

答案: C

程序中对象 a 是自动对象, main 结束时先调用派生类析构函数,再调用基类析构函数。
构造时 BASE(n+1) 中传入的是 'Y', 派生类成员 d 保存的是 'X', 所以析构输出顺序为:

XY

二、程序阅读题

1.

输出结果:

12121

构造函数、复制构造函数、赋值运算符中各自都有一个独立的局部静态变量 n。
执行过程如下:

1. A b, c; 调用两次默认构造, 分别输出 1、2
2. A d = b, e = c; 调用两次复制构造, 分别输出 1、2
3. b = e; 调用一次赋值运算符, 输出 1

连起来就是 12121。

2.

输出结果:

01234

对象 x 中先加入 0 到 9 共 10 个元素, 随后调用 5 次 Del()。虽然 Del() 的返回写法有些问题, 但这里返回值并没有被使用, 而 Num 经过 5 次递减后变成 5。因此输出运算符只会输出前 5 个元素, 也就是 0 1 2 3 4 连在一起的结果。

3.

输出结果:

987655308

这个类本质上是在用字符数组模拟高精度整数, 并且把低位放在前面。

- x(3, "789") 实际表示数值 987
- y(9, "123456789") 实际表示数值 987654321

相加得到：

987654321 + 987 = 987655308

输出运算符从高位向低位逆序输出，因此最终打印 987655308。

4.

输出结果：

None-0

White-0

Brown-0

这里要分清虚函数和非虚函数：

1. getColor() 在基类中是虚函数，因此通过基类引用调用时会发生动态绑定。
2. getWeight() 在基类中不是虚函数，所以即使派生类里写了同名函数，通过基类引用调用时仍然绑定到基类版本。

因此：

- a1 是普通 Animal 对象，输出 None-0
- a2 绑定到 Sheep，颜色走虚函数得到 White，重量仍调用基类版本得到 0
- a3 绑定到 Horse，颜色为 Brown，重量仍是 0

三、程序填空题

1.

填空答案：

(1.1) A&

(1.2) friend A

(1.3) return os

(1.4) 100

说明如下：

1. operator+= 应返回当前对象引用，便于连写 d += b += c
2. operator+ 是非成员友元函数，返回一个新的 A 对象
3. 输出运算符最后应返回流对象
4. 100 + c 中，100 可以借助构造函数隐式转换为 A(100)，因此结果为 103

验证一下主程序：

- b += c 后，b = 5
- d += b 后，d = 5
- d = 100 + c 后，d = 103

最终输出：

2.

填空答案:

- (2.1) `this != &w`
- (2.2) `this->w = w`
- (2.3) `w.getStyle()`
- (2.4) `Window("Rectangle")`

说明如下:

1. 赋值运算符中要先检查是否为自我赋值
2. House 中的数据成员和构造函数形参同名, 因此要用 `this->w` 指明成员对象
3. `getWindowStyle()` 直接返回成员对象 `w` 的风格字符串
4. 主函数中传入一个临时 Window 对象即可

这一题使用了 `char*` 和字符串字面量。在较新的 C++ 编译环境中, 更推荐把相关类型写成 `const char*`, 这样更规范。

四、编程题

1. 模板函数实现冒泡排序

这一题主要考察函数模板和数组上的通用排序思想。题目中的 `sort(10, a)` 与 `sort(5, c)` 表明函数应当能处理不同元素类型, 只要这些元素支持大小比较即可。

参考实现如下:

```
#include <iostream>
using namespace std;

template <class T>
void sort(int n, T a[])
{
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - 1 - i; ++j) {
            if (a[j + 1] < a[j]) {
                T temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

程序先比较相邻元素，若次序不对就交换。每一趟都会把当前最大的元素“冒”到后面，因此称为冒泡排序。由于用了模板，这个函数既能给 int 数组排序，也能给 char 数组排序。

2. Desktop、Leg 和 Table

这一题主要考察类的定义、组合关系、构造函数、访问器函数以及输出运算符重载。题意已经说明：一张桌子由一个桌面和四条桌腿组成，所以 Table 可以把 Desktop 作为一个成员对象，把四条 Leg 保存在容器中。

参考实现如下：

```
#include <iostream>
#include <vector>
using namespace std;

class Desktop {
public:
    Desktop(double l = 0, double w = 0, const char* c = "")
        : length(l), width(w), color(c) {}

    double getLength() const { return length; }
    double getWidth() const { return width; }
    const char* getColor() const { return color; }

private:
    double length;
    double width;
    const char* color;
};

class Leg {
public:
    Leg(double h = 0, const char* c = "")
        : height(h), color(c) {}

    double getHeight() const { return height; }
    const char* getColor() const { return color; }

private:
    double height;
    const char* color;
};

class Table {
public:
```

```

Table(const Desktop& t, const Leg ls[]) : top(t) {
    for (int i = 0; i < 4; ++i)
        legs.push_back(ls[i]);
}

friend ostream& operator<<(ostream& os, const Table& t) {
    os << "This table has a " << t.top.getColor() << " "
        << t.top.getLength() << "*" << t.top.getWidth()
        << " desktop." << endl;
    os << "Its legs are as follow:" << endl;
    for (int i = 0; i < 4; ++i) {
        os << t.legs[i].getColor() << " "
            << t.legs[i].getHeight() << endl;
    }
    return os;
}

private:
    Desktop top;
    vector<Leg> legs;
};

```

这份写法里，Table 通过组合保存自己的部件。构造函数负责把桌面和四条桌腿复制进来，输出运算符则按题目要求组织输出格式。若严格对样例，主函数运行后就会打印题目给出的那几行结果。